



Data & AI Workshop Assessment

Student Team Challenge | Thammasat University Public-Data Scenario | Tool-Agnostic

Assessment Goal

Design and demonstrate a small but production-minded Data & AI solution. The solution must ingest the provided Excel file, prepare logical data assets, produce trusted analytics output, and use the same Excel file as the Retrieval-Augmented Generation (RAG) document source.

Important constraint: this assessment is tool-agnostic. Teams may choose any programming language, orchestration approach, database, file format, vector store, local LLM, or LLM API. The review focuses on design quality, re-run safety, clear data specification, quality summary, analytics usability, and AI/RAG explanation.

Batch constraint: do not hard-code the process to only one fixed daily schedule. Design the batch as a reusable parameterized process with `business_date`, `run_id`, `input_file_name`, `sheet_name`, and `input_file_checksum`. A daily run is one possible schedule, but the same design should support re-run, backfill, or ad hoc batch execution.

For this version, the provided workbook is synthetic/mock workshop data. Students should use the values as provided. Do not create masked, hashed, or encrypted output columns for this Excel file.

Input File Provided

File	Purpose	Important Notes
thammasat_workshop_dataset_one_sheet_rag_behavior_salary.xlsx	Main dataset for ingestion, profiling, quality summary, analytics output, behavior/salary mock-up, and RAG document corpus.	Only one sheet: <code>workshop_data</code> . The instructor will not provide additional source files, databases, orchestrators, accounts, keys, websites, or infrastructure. Teams must work from this file and their own selected tools.

The workbook currently contains 180 rows and 49 columns. The rows are synthetic training records created for education. They are not real Thammasat student records or official salary data.

Recommended Team Positions

Position	Suggested Ownership
Project Manager	Coordinate team workflow, split tasks, track assumptions, keep the submission consistent, and make sure every part connects end-to-end.
Business Analyst (Data Product)	Clarify the use case, define expected data products, write business definitions, align outputs with user questions, and check whether the analytics/RAG result is understandable.
Data Engineer	Own ingestion, idempotent batch behavior, schema handling, row counts, audit log, dedupe/upsert, and quality summary.
Data Analyst / Data Modeler	Own profiling, data specification, business keys, source-to-target mapping, and analytics/visualization output.
Data Governance / Security Engineer	Own classroom data-handling assumptions, column-level usage classification, access assumptions, and safe RAG rules.
AI Engineer	Own document chunking, retrieval, prompt design, local LLM or API usage, source citation, and RAG evaluation.

Teamwork note: These positions are suggested responsibilities only. There is no Data Tech Lead role for this student workshop. All students should help each other, review each part together, and be able to explain the full solution end-to-end.

Required Output from Each Team

Deliverable	Minimum Content Expected
1. Data platform architecture overview	Diagram plus short explanation of ingestion, storage/layers, transformation, serving, governance, observability, and AI/RAG components.
2. Data specification	Schema, business keys, technical metadata, source-to-target mapping, partition/snapshot strategy, quality summary design, and column-level usage classification for the current Excel.
3. Idempotent batch pipeline	Reusable batch ingestion flow, duplicate prevention, re-run behavior, audit log, error handling, retry strategy, and quality summary checks.
4. Trusted analytics output	Final trusted table/view/file and at least one simple visualization or analysis view. Use the workbook values as provided; no masked/hashed/encrypted output is required for this workshop Excel.
5. RAG and visualization proof of concept	Use rag_document_title, rag_document_text, rag_keywords, rag_sample_question, and rag_expected_answer_hint from the same Excel file. Retrieve relevant chunks, call local LLM or API if available, answer with source references, and present results through an app, dashboard, BI report, notebook, report page, or chatbot-style UI.

Part 0 - Architecture Overview First

Start with the platform design before showing code. The diagram should make it clear how data moves from the source Excel file to trusted analytics and then to RAG.

- Show source Excel file, landing/raw layer, staged/typed layer, trusted/model layer, serving layer, and AI/RAG path.
- Show where orchestration, logging, quality summary, metadata/catalog, lineage, access assumptions, and monitoring exist.
- Show the RAG path separately: Excel RAG fields, chunking, embedding or keyword index, retrieval, prompt, LLM response, and source citation.
- Explain trade-offs: batch cadence, full refresh vs incremental/upsert, file vs table storage, local LLM vs LLM API, and how the design changes if the volume grows.

Recommended position: Project Manager coordinates the architecture story. Business Analyst (Data Product) confirms the target data products and user questions. Data Engineer and AI Engineer support the data flow and RAG path.

Hint / Solution Direction: A strong answer separates platform layers and responsibilities. Do not draw only one script connected to one database. Show control flow, data flow, quality checkpoints, and where RAG reads from the Excel-derived document corpus.

Part 1 - Data Discovery and Specification

Use the one-sheet Excel file. The current version contains student-like synthetic records plus behavior, career, salary expectation, and RAG fields in the same sheet. Do not spend time building complex validation rules. Focus on a clear specification, profiling, and quality summary.

- Profile row counts by record_type, campus, level, status, program_id, source_system, snapshot_date, career_interest, learning_preference, and teamwork_style where available.
- Create a data dictionary for student profile fields, academic fields, behavior/career/salary fields, RAG document fields, source metadata, and operational batch metadata.
- Define business keys such as entity_id or student_no for the student entity and source_row_no for row-level traceability.
- Define technical metadata such as load_timestamp, business_date, source_file_name, input_file_checksum, run_id, and record_status.
- Define target layer/table/file naming convention. Example: raw_workshop_data, stg_student_snapshot, trusted_student_snapshot, analytics_student_summary, rag_excel_document_chunk, batch_audit.

Data quality scope: Keep it simple. Use quality summary checks such as source row count vs loaded row count, count by record_type/campus/status/career_interest, sum/min/max of GPA, sum/min/max of credit_earned, sum/min/max of expected_salary_thb, min/max of snapshot_date, and equality between source summary and target summary.

Recommended position: Business Analyst (Data Product) and Data Analyst / Data Modeler co-own the specification. Data Engineer supports profiling and quality summary implementation. Data Governance / Security Engineer owns classroom data-handling assumptions. Project Manager checks that the spec is complete and consistent.

Column-Level Usage Specification for This Excel

Include this decision inside the data specification. This version of the workbook is synthetic/mock and prepared for the workshop. Use fields as provided; do not create transformed privacy columns for this exercise.

Column / Data Element	Class in this workbook	Expected Treatment in Spec
record_type, entity_id, source_row_no	Traceability / row identity	Use for routing, lineage, source evidence, and rerun checks. Keep as provided.
student_no, citizen_id, email, mobile, student_name	Synthetic student-like profile fields	Use as mock workshop fields. Keep as provided. No masked/hashed/encrypted output required in this exercise.
level, faculty_or_school, program_id, program_name, discipline_cluster, campus, year_of_study, admission_channel, status, is_international	Academic attributes	Use for dimensions, grouping, filtering, and analytics summaries.
gpa, credit_earned	Academic numeric measures	Use for quality summary and analytics. Compare count/sum/min/max between source and target.
behavior_profile, teamwork_style, learning_preference, career_interest, internship_interest, mock_interview_note	Behavior and career mock-up	Use for mock data product questions, visualization, and RAG examples. Treat as synthetic workshop data.
expected_salary_thb, salary_expectation_note	Expected salary mock-up	Use for analytics and RAG examples. State clearly that it is mock data, not official salary data.
rag_document_title, rag_document_text, rag_keywords, rag_sample_question, rag_expected_answer_hint	RAG-ready text fields	Use directly as the RAG corpus. Keep source_row_no/entity_id/student_no as evidence metadata.
batch_date, snapshot_date, source_system, source_url, task_hint	Operational and source metadata	Use for audit, lineage, citation, and run explanation.

Part 2 - Idempotent Batch Pipeline

Build or design a batch pipeline that can be executed repeatedly without creating duplicate final records. The pipeline should be parameterized, not hard-coded to one date or one fixed schedule.

Requirement	Hint / Solution Direction
Parameterized batch	Accept business_date, run_id, input file path/name, sheet_name, and input file checksum. A daily schedule is acceptable, but the pipeline must also support re-run, backfill, and ad hoc batch execution.
Idempotency	Running the same input file and same business date twice should produce the same final result. Use input checksum, business key, snapshot_date, and overwrite/merge strategy.
Batch control	Create run log with run_id, business_date, start_time, end_time, status, input_file_name, sheet_name, input_file_checksum, source_count, loaded_count, rejected_count, and error_message.
Layering	Preserve raw rows. Convert and standardize types in staging. Deduplicate and publish in trusted/model layer.
Upsert / replacement logic	Merge by entity_id or student_no + snapshot_date, or replace the target business_date/snapshot atomically. Explain why the chosen strategy is safe.
Failure behavior	A failed run can be retried safely. Use staging area, temporary output, transaction, checkpoint, or run status to prevent partial final output.
Quality summary	Compare source summary vs loaded/trusted summary: count, sum, min, max. Failed equality checks should stop or flag the run.

Recommended position: Data Engineer owns the batch design and implementation. Project Manager tracks run evidence. Business Analyst (Data Product) confirms the output supports the requested data product. Other members review the re-run pattern together.

Hint / Solution Direction: Use a small control table or control file. For a simple implementation, read the Excel, calculate input checksum, write raw, create staged typed data, then publish trusted output using overwrite-by-business_date or merge-by-key. Re-run the same input twice and show that final counts do not double.

Part 3 - Data Handling and Access Design

Classify fields and state access assumptions. For this specific Excel file, do not implement masking, hashing, or encryption. The workbook is synthetic/mock and prepared for workshop use. The expected output is a clear usage/access decision, not a transformed privacy dataset.

Control Area	Hint / Solution Direction
Workshop data handling	Use the workbook values as provided. State that the dataset is synthetic/mock and not official student records.
Access levels	Define who can view raw, trusted, analytics, and RAG outputs in the workshop context. For production, describe that real student data would require stricter controls, but do not implement them here.
Analytics safety	Use row-level and aggregate analysis freely for this mock dataset. Label salary and behavior columns as mock-up.
RAG safety	Use the Excel RAG fields as the corpus. Answers should cite source_row_no/entity_id/student_no or source_url when available. Do not guess beyond the workbook.
No privacy transformation required	Do not create masked columns, hashed identifiers, encrypted output, or reconstructed values. Keep the current Excel fields unchanged.

Recommended position: Data Governance / Security Engineer owns the usage matrix. Business Analyst (Data Product) confirms access assumptions. Data Engineer keeps outputs unchanged. AI Engineer confirms the RAG corpus uses the Excel fields correctly. All members should understand what would change for real sensitive data.

Hint / Solution Direction: The key deliverable is a column usage matrix for this workbook. Students should be able to explain that the exercise data is safe for workshop use, while real personal data would need stronger governance outside the scope of this build.

Part 4 - RAG Question Answering from the Current Excel

Use the columns `rag_document_title`, `rag_document_text`, `rag_keywords`, `rag_sample_question`, and `rag_expected_answer_hint` as the document corpus. Build a simple RAG design or working prototype, then present the result through any suitable visualization or user-facing layer. This can be an app, dashboard, BI report, notebook, or chatbot-style interface; it is not limited to a chatbot app.

Step	Hint / Solution Direction
Document preparation	For rows with <code>rag_document_text</code> , create <code>document_id</code> or <code>chunk_id</code> using <code>source_row_no/entity_id</code> . Keep <code>rag_document_title</code> , <code>rag_keywords</code> , <code>source_url</code> , <code>student_no</code> , and <code>source_row_no</code> as metadata.
Embedding / retrieval	Use any embedding model, vector store, or keyword search. Explain choice and limitation. Keyword retrieval is acceptable if no model is available.
Prompting	Ask the model to answer only from retrieved Excel context and include source references. Include instruction for not-found answers.
LLM option	Use local LLM or LLM API if available. If not available, show top retrieved rows/chunks and the final prompt that would be sent.
Evaluation	Test several questions and label each answer as grounded, partially grounded, or not found. Show retrieved source rows.
Visualization / presentation	Show the RAG result in any clear format: simple app, BI/dashboard page, notebook output, report page, or chatbot-style UI. Include answer, source reference, and confidence/grounding status where possible.

Recommended position: AI Engineer owns the RAG flow. Business Analyst (Data Product) designs useful questions, checks answer quality, and helps define visualization/user experience. Data Analyst / Data Modeler may support BI/dashboard output. Data Governance / Security Engineer checks usage assumptions. Data Engineer supports corpus preparation. Project Manager keeps the demo flow clear.

Hint / Solution Direction: A complete RAG answer is not only an LLM response. It should show the retrieved context, the prompt pattern, answer, source_row_no/source_url citation, and what happens when the answer is not in the workbook. Choose the fastest and clearest interface for the team.

Suggested RAG Questions

- Which students are interested in Data Engineer, AI Engineer, or analytics-related careers?
- What is the expected salary range from the mock salary expectation column?
- Which teamwork styles and learning preferences appear in the dataset?
- For a selected `student_no`, what behavior profile and career interest are described?
- Which rows support the answer, and what `source_row_no` or `source_url` should be cited?
- What should the system answer when the workbook does not contain evidence for the question?

Submission Checklist

- Architecture diagram image or slide based on the current Excel workbook.
- README or short document explaining setup, run steps, assumptions, and limitations.
- Code or pseudocode for ingestion, transformation, quality summary, analytics output, and RAG steps.
- Data specification including schema, keys, technical metadata, source-to-target mapping, and column usage matrix for the current Excel.
- Evidence of idempotent re-run: before/after row counts or run logs showing the final result does not duplicate.
- Quality summary evidence: count, sum, min, max comparison between source file and target layer.
- RAG examples with retrieved chunks/rows, answer, source references, and a simple visualization/interface view such as app, BI/dashboard, notebook, report page, or chatbot-style UI.
- No masked, hashed, or encrypted output columns created for this workshop Excel.

Hint / Solution Direction: The submission does not need to be perfect production code. It must be understandable, runnable or explainable, and show the reasoning behind design decisions.

Public Source Notes for Dataset Context

The workbook includes `source_url` values in the single sheet so students can cite context during analysis. The exercise data is for education only and must not be treated as official student-level records, official salary data, or an official university dataset.

End of assessment brief.